

API REST de Alunos

Cadastro, consulta, atualização e exclusão de alunos com FastAPI, persistência no MongoDB e testes no Postman.

Python

FastAPI

Pydantic

MongoDB

PyMongo

Postman

INTEGRANTES

Bruno Henrique Martins Ferreira

RM566277

Anderson Huang

RM565920

Guyllherme Miguel

RM562374

Sumário

- 1 Objetivo e contexto
- 2 Tecnologias e arquitetura
- 3 Estrutura do projeto
- 4 Como executar
- 5 Endpoints da API
- 6 Regras de validação
- 7 Banco de dados
- 8 Evidências de funcionamento
- 9 Testes no Postman
- 10 Testes automatizados
- 11 Atendimento aos requisitos

1 Objetivo e contexto

Uma instituição de ensino precisa de uma API para gerenciar seus alunos. Este projeto implementa uma API REST em Python com FastAPI, integrada a um banco de dados MongoDB, que permite cadastrar, listar, consultar, atualizar e excluir alunos.

Cada aluno tem os atributos `nome`, `email`, `idade` e `curso`, e é armazenado como um documento na coleção `alunos` do banco `escola`. Os testes da API foram feitos com uma collection do Postman.

2 Tecnologias e arquitetura

FastAPI + Uvicorn

Framework web e servidor ASGI que expõem os endpoints REST e a documentação em `/docs`.

Pydantic

Validação dos dados recebidos: campos obrigatórios, formato de e-mail e idade mínima.

MongoDB + PyMongo

Banco NoSQL orientado a documentos e driver oficial para Python.

Fluxo de uma requisição



A requisição chega ao FastAPI, o corpo é validado pelo Pydantic e só dados válidos são enviados ao MongoDB. Erros de validação voltam como `422`, e-mail repetido como `409` e ID inexistente como `404`.

3 Estrutura do projeto

```
cp4-cloud-database/
├── app/
│   ├── main.py          # endpoints e tratamento de erros
│   ├── schemas.py       # modelos Pydantic e regras de validação
│   └── database.py       # conexão com o MongoDB (escola/alunos)
├── postman/
│   └── CP4-API-Alunos.postman_collection.json
├── evidencias/          # prints das execuções (figuras deste documento)
├── tests/test_api.py    # testes automatizados (pytest + mongomock)
├── docker-compose.yml   # MongoDB local
├── requirements.txt
├── requirements-dev.txt
└── .env.example         # MONGO_URI (local ou Atlas) e MONGO_DB
```

4 Como executar

1. Subir o MongoDB (escolha uma opção)

```
docker compose up -d # MongoDB 7 local na porta 27017
# ou: MongoDB instalado localmente, ou MongoDB Atlas (nuvem):
export MONGO_URI="mongodb+srv://usuario:senha@cluster.mongodb.net"
```

2. Instalar as dependências

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3. Iniciar a API

```
uvicorn app.main:app --reload
# API: http://localhost:8000
# Swagger: http://localhost:8000/docs
```

Sem configuração, a API usa `mongodb://localhost:27017` e o banco `escola`. Ao iniciar, ela cria o índice único `uk_email` na coleção `alunos`.

5 Endpoints da API

MÉTODO	ROTA	DESCRIÇÃO	SUCESSO	ERROS
POST	/alunos	Cadastra um aluno	201	422 409
GET	/alunos	Lista todos os alunos	200	—
GET	/alunos/{id}	Consulta um aluno pelo ID	200	404
PUT	/alunos/{id}	Atualiza um aluno	200	404 409 422
DELETE	/alunos/{id}	Remove um aluno	200	404

Exemplo de cadastro

```
POST /alunos
{
  "nome": "João Silva",
  "email": "joao@email.com",
  "idade": 22,
  "curso": "Ciência da Computação"
}
```

Resposta **201 Created** :

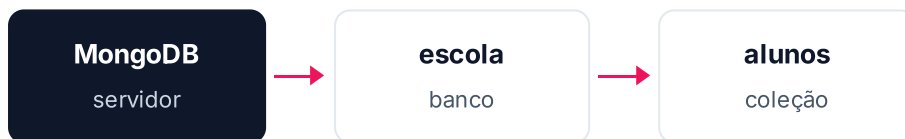
```
{
  "mensagem": "Aluno cadastrado com sucesso",
  "aluno": { "id": "68c123...", "nome": "João Silva", "email": "joao@email.com",
    "idade": 22, "curso": "Ciência da Computação" }
}
```

6 Regras de validação

CAMPO	REGRA	COMO FOI IMPLEMENTADO
nome	Não pode ser vazio	<code>field_validator</code> que remove espaços e recusa texto vazio
email	Formato válido	Tipo <code>EmailStr</code> do Pydantic
idade	Maior ou igual a 16	<code>Field(ge=16)</code>
curso	Não pode ser vazio	Mesmo validador do campo nome
email	Não pode repetir	Verificação na API (409) e índice único <code>uk_email</code> no MongoDB. O e-mail é salvo em minúsculas.

Uma requisição inválida retorna `422` com todos os erros de uma vez, um por campo (ver Figura 7).

7 Banco de dados



Cada aluno é um documento da coleção `alunos`. A API converte o `_id` (ObjectId) para o campo `id` nas respostas.

```
{
  "_id": ObjectId("..."),
  "nome": "João Silva",
  "email": "joao@email.com",
  "idade": 22,
  "curso": "Ciência da Computação"
}
```

Índices da coleção: `_id_`, padrão do MongoDB, e `uk_email` (`{ email: 1 }`, `unique: true`), que impede e-mails duplicados mesmo com requisições simultâneas.

8 Evidências de funcionamento

As evidências abaixo vêm de chamadas reais à API em 29/09/2026, com a aplicação rodando em `localhost:8000` e conectada a um MongoDB 7. Cada print mostra a requisição enviada e a resposta recebida (status, tempo e corpo).

The screenshot displays the Swagger UI for an API titled "API de Alunos". At the top, it shows the version "1.0.0" and the specification "OAS 3.1". Below the title, a link to the OpenAPI JSON file is provided. A descriptive text states: "CP4 - Cloud Database: CRUD de alunos com FastAPI e MongoDB."

The interface is organized into sections. The "Health" section contains a single endpoint: a GET request to the root path ("/ Raiz").

The "Alunos" section lists five endpoints for managing students:

- GET /alunos: Listar Alunos
- POST /alunos: Cadastrar Aluno
- GET /alunos/{id}: Consultar Aluno
- PUT /alunos/{id}: Atualizar Aluno
- DELETE /alunos/{id}: Excluir Aluno

The "Schemas" section defines the data models used in the API:

- AlunoCreate: object
- AlunoResponse: object
- AlunoUpdate: object
- HTTPValidationError: object
- Mensagem: object
- ValidationError: object

Figura 1. Documentação interativa (Swagger UI) em /docs

Documentação gerada pelo FastAPI com os cinco endpoints de alunos e os schemas de validação.

1. Cadastrar aluno	
POST http://localhost:8000/alunos	
REQUISIÇÃO (BODY)	RESPOSTA 201 CREATED 27 MS
<pre>{ "nome": "João Silva", "email": "joao@email.com", "idade": 22, "curso": "Ciência da Computação" }</pre>	<pre>{ "mensagem": "Aluno cadastrado com sucesso", "aluno": { "nome": "João Silva", "email": "joao@email.com", "idade": 22, "curso": "Ciência da Computação", "id": "6abc509eeb4574e10193e7a6" } }</pre>

Figura 2. POST /alunos: cadastro de aluno

O aluno é gravado no MongoDB e a API responde 201 Created com a mensagem de sucesso e o id gerado.

2. Cadastrar segundo aluno	
POST http://localhost:8000/alunos	
REQUISIÇÃO (BODY)	RESPOSTA 201 CREATED 4 MS
<pre>{ "nome": "Maria Santos", "email": "maria@email.com", "idade": 24, "curso": "Engenharia de Software" }</pre>	<pre>{ "mensagem": "Aluno cadastrado com sucesso", "aluno": { "nome": "Maria Santos", "email": "maria@email.com", "idade": 24, "curso": "Engenharia de Software", "id": "6abc509eeb4574e10193e7a7" } }</pre>

Figura 3. POST /alunos: segundo cadastro

Um segundo aluno é cadastrado para demonstrar a listagem com mais de um registro.

3. Listar todos os alunos	
GET http://localhost:8000/alunos	
REQUISIÇÃO (BODY) - sem corpo -	RESPOSTA 200 OK 5 MS <pre>[{ "nome": "João Silva", "email": "joao@email.com", "idade": 22, "curso": "Ciência da Computação", "id": "6abc509eeb4574e10193e7a6" }, { "nome": "Maria Santos", "email": "maria@email.com", "idade": 24, "curso": "Engenharia de Software", "id": "6abc509eeb4574e10193e7a7" }]</pre>

Figura 4. GET /alunos: listagem de todos os alunos

Retorna a lista de alunos cadastrados, com o _id do MongoDB convertido para o campo id.

4. Consultar aluno por ID	
GET http://localhost:8000/alunos/6abc509eeb4574e10193e7a6	
REQUISIÇÃO (BODY) - sem corpo -	RESPOSTA 200 OK 5 MS <pre>{ "nome": "João Silva", "email": "joao@email.com", "idade": 22, "curso": "Ciência da Computação", "id": "6abc509eeb4574e10193e7a6" }</pre>

Figura 5. GET /alunos/{id}: consulta de um aluno específico

Retorna somente o aluno correspondente ao ID informado.

5. Atualizar aluno	
PUT http://localhost:8000/alunos/6abc509eeb4574e10193e7a6	
REQUISIÇÃO (BODY) <pre>{ "nome": "João da Silva", "email": "joao.silva@email.com", "idade": 23, "curso": "Engenharia de Software" }</pre>	RESPOSTA 200 OK 5 MS <pre>{ "mensagem": "Aluno atualizado com sucesso", "aluno": { "nome": "João da Silva", "email": "joao.silva@email.com", "idade": 23, "curso": "Engenharia de Software", "id": "6abc509eeb4574e10193e7a6" } }</pre>

Figura 6. PUT /alunos/{id}: atualização

Nome, e-mail, idade e curso são atualizados no MongoDB e a API devolve o registro atualizado.

6. Requisição inválida (regras de validação)

POST http://localhost:8000/alunos

REQUISIÇÃO (BODY)

```
{
  "nome": "",
  "email": "email_invalido",
  "idade": 12,
  "curso": ""
}
```

RESPOSTA

422 UNPROCESSABLE CONTENT 4 MS

```
{
  "detail": "Erro de validação",
  "erros": [
    {
      "campo": "nome",
      "mensagem": "não pode ser vazio"
    },
    {
      "campo": "email",
      "mensagem": "value is not a valid email address: An email address must have an @-sign."
    },
    {
      "campo": "idade",
      "mensagem": "Input should be greater than or equal to 16"
    },
    {
      "campo": "curso",
      "mensagem": "não pode ser vazio"
    }
  ]
}
```

Figura 7. Regras de validação: requisição inválida

Nome vazio, e-mail inválido, idade menor que 16 e curso vazio geram 422 com a lista de erros por campo.

7. E-mail duplicado

POST http://localhost:8000/alunos

REQUISIÇÃO (BODY)

```
{
  "nome": "Outra Maria",
  "email": "maria@email.com",
  "idade": 30,
  "curso": "Sistemas de Informação"
}
```

RESPOSTA

409 CONFLICT 4 MS

```
{
  "detail": "Já existe um aluno cadastrado com este e-mail"
}
```

Figura 8. Regra de e-mail único

Uma tentativa de cadastro com e-mail já existente é recusada com 409 Conflict.

```
mongosh escola - coleção alunos

escola> db.alunos.find()
[
  {
    _id: ObjectId('6abc509eeb4574e10193e7a6'),
    nome: 'João da Silva',
    email: 'joao.silva@email.com',
    idade: 23,
    curso: 'Engenharia de Software'
  },
  {
    _id: ObjectId('6abc509eeb4574e10193e7a7'),
    nome: 'Maria Santos',
    email: 'maria@email.com',
    idade: 24,
    curso: 'Engenharia de Software'
  }
]

escola> db.alunos.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { email: 1 }, name: 'uk_email', unique: true }
]
```

Figura 9. MongoDB: banco escola, coleção alunos

Consulta direta no mongosh: os documentos gravados pela API e o índice único uk_email no campo email.

9. Excluir aluno

DELETE http://localhost:8000/alunos/6abc509eeb4574e10193e7a6

REQUISIÇÃO (BODY)	RESPOSTA 200 OK 5 MS
- sem corpo -	<pre>{ "mensagem": "Aluno removido com sucesso", "aluno": null }</pre>

Figura 10. DELETE /alunos/{id}: exclusão

O aluno é removido do MongoDB e a API confirma a operação.

10. Consultar aluno excluído

GET http://localhost:8000/alunos/6abc509eeb4574e10193e7a6

REQUISIÇÃO (BODY)	RESPOSTA 404 NOT FOUND 4 MS
- sem corpo -	<pre>{ "detail": "Aluno não encontrado" }</pre>

Figura 11. GET /alunos/{id} de aluno inexistente

Após a exclusão, a consulta ao mesmo ID retorna 404 Not Found.

11. Excluir aluno inexistente

DELETE http://localhost:8000/alunos/6abc509eeb4574e10193e7a6

REQUISIÇÃO (BODY)	RESPOSTA
<pre>- sem corpo -</pre>	404 NOT FOUND 4 MS <pre>{ "detail": "Aluno não encontrado" }</pre>

Figura 12. DELETE /alunos/{id} de aluno inexistente
A API informa que o registro não foi encontrado (404).

```
mongosh escola - após o DELETE

escola> db.alunos.find({}, {nome: 1, email: 1})
[
  {
    _id: ObjectId('6abc509eeb4574e10193e7a7'),
    nome: 'Maria Santos',
    email: 'maria@email.com'
  }
]

escola> db.alunos.countDocuments()
1
```

Figura 13. MongoDB após o DELETE
Somente o aluno não excluído permanece na coleção, confirmando a remoção no banco.

9 Testes no Postman

A collection `postman/CP4-API-Alunos.postman_collection.json` tem 10 requisições com testes automáticos de status. O ID do aluno cadastrado é salvo na variável `alunoId` e reaproveitado nas requisições seguintes.

PASSO	COMO FAZER
1	No Postman, clique em Import e selecione o arquivo da collection.
2	A variável <code>baseUrl</code> já vem como <code>http://localhost:8000</code> .
3	Execute as requisições em ordem ou use Run collection .

Para rodar a collection de novo, limpe a coleção `alunos` antes. Senão o cadastro da Maria retorna 409, porque o e-mail dela já existe.

```
newman run postman/CP4-API-Alunos.postman_collection.json

newman

CP4 - Cloud Database - API de Alunos

→ 1. Cadastrar aluno (João)
  POST http://localhost:8000/alunos [201 Created, 308B, 16ms]
  ✓ 201 Created

→ 2. Cadastrar aluno (Maria)
  POST http://localhost:8000/alunos [201 Created, 308B, 4ms]
  ✓ 201 Created

→ 3. Listar todos os alunos
  GET http://localhost:8000/alunos [200 OK, 379B, 3ms]
  ✓ 200 OK
  ✓ Retorna uma lista

→ 4. Consultar aluno por ID
  GET http://localhost:8000/alunos/6abc4e4ceb4574e10193e792 [200 OK, 251B, 4ms]
  ✓ 200 OK

→ 5. Atualizar aluno
  PUT http://localhost:8000/alunos/6abc4e4ceb4574e10193e792 [200 OK, 310B, 5ms]
  ✓ 200 OK
  ✓ Curso atualizado

→ 6. Requisição inválida (validação)
  POST http://localhost:8000/alunos [422 Unprocessable Content, 468B, 2ms]
  ✓ 422 Unprocessable Entity

→ 7. E-mail duplicado
  POST http://localhost:8000/alunos [409 Conflict, 190B, 2ms]
  ✓ 409 Conflict

→ 8. Excluir aluno
  DELETE http://localhost:8000/alunos/6abc4e4ceb4574e10193e792 [200 OK, 179B, 2ms]
  ✓ 200 OK

→ 9. Consultar aluno excluído (404)
  GET http://localhost:8000/alunos/6abc4e4ceb4574e10193e792 [404 Not Found, 166B, 2ms]
  ✓ 404 Not Found

→ 10. Excluir aluno inexistente (404)
  DELETE http://localhost:8000/alunos/6abc4e4ceb4574e10193e792 [404 Not Found, 166B, 2ms]
  ✓ 404 Not Found
```

	executed	failed
iterations	1	0
requests	10	0
test-scripts	10	0
prerequisite-scripts	0	0
assertions	12	0
total run duration: 159ms		
total data received: 1.42kB (approx)		
average response time: 4ms [min: 2ms, max: 16ms, s.d.: 4ms]		

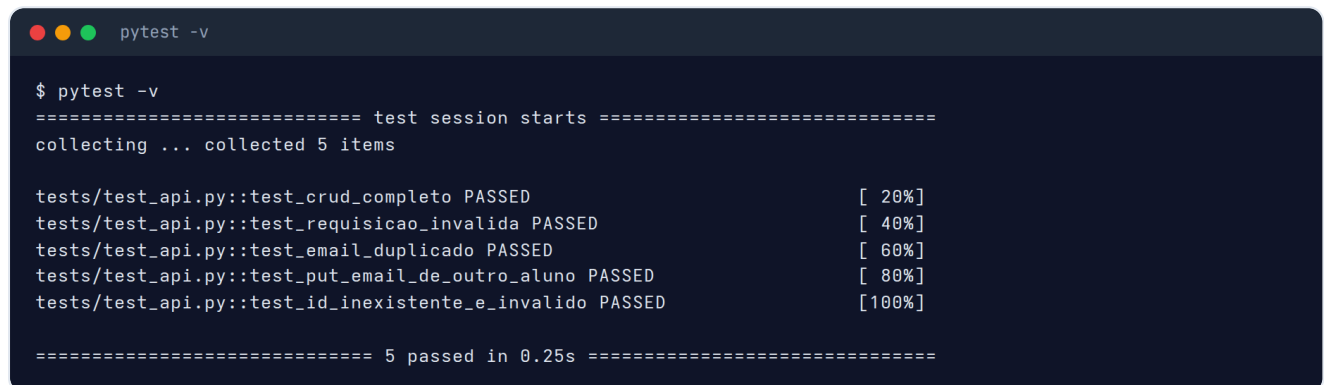
Figura 14. Execução da collection do Postman (Newman)

As 10 requisições da collection foram executadas em ordem: 12 asserções, nenhuma falha.

10 Testes automatizados

Além do Postman, o projeto tem testes com `pytest`. Eles usam `mongomock` e não precisam de um MongoDB rodando. Os testes cobrem o CRUD completo, a requisição inválida, o e-mail duplicado no cadastro e na atualização, e os IDs inexistentes ou inválidos.

```
pip install -r requirements-dev.txt
pytest -v
```



```
pytest -v

$ pytest -v
===== test session starts =====
collecting ... collected 5 items

tests/test_api.py::test_crud_completo PASSED [ 20%]
tests/test_api.py::test_requisicao_invalida PASSED [ 40%]
tests/test_api.py::test_email_duplicado PASSED [ 60%]
tests/test_api.py::test_put_email_de_outro_aluno PASSED [ 80%]
tests/test_api.py::test_id_inexistente_e_invalido PASSED [100%]

===== 5 passed in 0.25s =====
```

Figura 15. Testes automatizados (pytest)
Os 5 cenários automatizados passaram.

11 Atendimento aos requisitos

REQUISITO DA TAREFA	STATUS	EVIDÊNCIA
API REST com Python e FastAPI	Atendido	Seção 2 e Fig. 1
Banco <code>escola</code> , coleção <code>alunos</code>	Atendido	Fig. 9
<code>POST /alunos</code> com confirmação do cadastro	Atendido	Fig. 2 e 3
<code>GET /alunos</code> lista todos os alunos	Atendido	Fig. 4
<code>GET /alunos/{id}</code> , com 404 se não existir	Atendido	Fig. 5 e 11
<code>PUT /alunos/{id}</code> atualiza no MongoDB	Atendido	Fig. 6 e 9
<code>DELETE /alunos/{id}</code> , informando ID inexistente	Atendido	Fig. 10, 12 e 13
Validações: nome, e-mail, idade ≥ 16 , curso	Atendido	Fig. 7
Sem e-mails duplicados	Atendido	Fig. 8 e 9
Testes com Postman	Atendido	Fig. 14